
```
// Denis Loren Moldovan, Jorge Adrian Saghin Dudulea, Gr. 121
// 100522240@alumnos.uc3m.es, 100522257@alumnos.uc3m.es
%{
    // SECCION 1 Declaraciones de C-Yacc

#include <stdio.h>
#include <ctype.h>          // declaraciones para tolower
#include <string.h>         // declaraciones para cadenas
#include <stdlib.h>         // declaraciones para exit ()

#define FF fflush(stdout);  // para forzar la impresion inmediata
#define INC(x) x=x+1;
#define DEC(x) x=x-1;

int yylex () ;
int yyerror (char*) ;
char *mi_malloc (int) ;
char *gen_code (char *) ;
char *int_to_string (int) ;
char *char_to_string (char) ;

char temp [2048] ;

int en_funcion = 0;        // 0 si es variable global, 1 si esta dentro de una función
char nombre_funcion[256];  // Nombre de la función local
char *vars_locales[256];  // Tabla de nombres de variables locales
int n_vars_locales = 0;    // Número de variables locales
int nivel_rama = 0;        // Si nivel_rama en_rama == 0, se está fuera de un if/for/switch/while. Si > 0, indica que se está dentro de una rama
int nivel_en_return = 0;   // Guarda el valor de nivel_rama cuando se parsea el return

void insertar_local(char *nombre);
int es_local(char *nombre);
void limpiar_locales();
char *nombre_local(char *var);
char *wrap_progn(char *codigo);

// Abstract Syntax Tree (AST) Node Structure

typedef struct ASTnode t_node ;

struct ASTnode {
    char *op ;
    int type ;    // leaf, unary or binary nodes
    t_node *left ;
    t_node *right ;
} ;

// Definitions for explicit attributes

typedef struct s_attr {
    int value ;    // - Numeric value of a NUMBER
    char *code ;   // - to pass IDENTIFIER names, and other translations
    t_node *node ; // - for possible future use of AST
} t_attr ;

#define YYSTYPE t_attr

%}

// Definitions for explicit attributes
```

```

%token NUMBER
%token IDENTIF // Identificador=variable
%token INTEGER // identifica el tipo entero
%token STRING
%token PUTS
%token PRINTF
%token MAIN // identifica el comienzo del proc. main
%token OR
%token AND
%token NEQ
%token EQ
%token LTEQ
%token GTEQ
%token FOR
%token WHILE // identifica el bucle main
%token IF
%token INC
%token DEC
%token ELSE
%token SWITCH
%token CASE
%token BREAK
%token DEFAULT
%token RETURN

%right '=' // es la ultima operacion que se debe realizar
%left OR
%left AND
%left EQ NEQ
%left '>' '<' LTEQ GTEQ
%left '+' '-' // menor orden de precedencia
%left '*' '/' '%' // orden de precedencia intermedio
%right '!' UNARY_SIGN // mayor orden de precedencia

%% // Seccion 3 Gramatica - Semantico

axioma:  declaraciones_funciones funcion_main { ; }
;

declaraciones_funciones: decl_global ';' { printf("%s\n", $1.code) ; } declaraciones_funciones { $$code = gen_code("") ; }
| funcion { printf("%s\n", $1.code) ; } declaraciones_funciones { $$code = gen_code("") ; }
|
;

decl_global: INTEGER IDENTIF { sprintf(temp, "(setq %s 0)", $2.code) ;
    $$code = gen_code(temp) ; }
| INTEGER IDENTIF '=' NUMBER { sprintf(temp, "(setq %s %d)", $2.code, $4.value) ;
    $$code = gen_code(temp) ; }
| INTEGER IDENTIF '=' NUMBER mult_assign { sprintf(temp, "(setq %s %d) %s", $2.code, $4.value, $5.code) ;
    $$code = gen_code(temp) ; }
| INTEGER IDENTIF mult_assign { sprintf(temp, "(setq %s 0) %s", $2.code, $3.code) ;
    $$code = gen_code(temp) ; }
| INTEGER IDENTIF '[' NUMBER ']' { sprintf(temp, "(setq %s (make-array %d))", $2.code, $4.value) ;
    $$code = gen_code(temp) ; }
;

funcion: IDENTIF '(' lista_args ')'
{ strcpy(nombre_funcion, $1.code) ;
  limpiar_locales();
  en_funcion = 1;}

'{' cuerpo
'}'

{en_funcion = 0;

```

```

        sprintf(temp, "(defun %s (%s)\n%s)", $1.code, $3.code, $7.code) ;
        $$code = gen_code(temp) ; }

;

lista_args: INTEGER IDENTIF r_lista_args
    { sprintf(temp, "%s %s", $2.code, $3.code) ;
      $$code = gen_code(temp) ; }
    |
    { $$code = gen_code("") ; }
;

r_lista_args: ',' INTEGER IDENTIF r_lista_args
    { sprintf(temp, "%s %s", $3.code, $4.code) ;
      $$code = gen_code(temp) ; }
    |
    { $$code = gen_code("") ; }
;

funcion_main: MAIN '(' ')'

    { strcpy(nombre_funcion, "main") ;
      limpiar_locales();
      en_funcion = 1; }

    '{' cuerpo '}'

    { en_funcion = 0;
      sprintf(temp, "(defun main ()\n%s)", $6.code) ;
      $$code = gen_code(temp) ;
      printf("%s\n", $$code) ; }

    |
    { $$code = gen_code("") ;
      printf("%s\n", $$code) ; }
;

cuerpo: sentencia ';' cuerpo

    { if (strlen($3.code) > 0) {
        sprintf(temp, "%s\n%s", $1.code, $3.code) ;
      } else {
        sprintf(temp, "%s", $1.code) ;
      }
      $$code = gen_code(temp) ; }

    | bucle_while cuerpo
    { sprintf(temp, "%s\n%s", $1.code, $2.code) ;
      $$code = gen_code(temp) ; }

    | control_if cuerpo
    { sprintf(temp, "%s\n%s", $1.code, $2.code) ;
      $$code = gen_code(temp) ; }

    | bucle_for cuerpo
    { sprintf(temp, "%s\n%s", $1.code, $2.code) ;
      $$code = gen_code(temp) ; }

    | control_switch cuerpo
    { sprintf(temp, "%s\n%s", $1.code, $2.code) ;
      $$code = gen_code(temp) ; }

    | RETURN expresion ';' { nivel_en_return = nivel_rama ; } cuerpo
    { if (strlen($5.code) > 0 || nivel_en_return > 0) {
        if (strlen($5.code) > 0) {
            sprintf(temp, "(return-from %s %s)\n%s", nombre_funcion, $2.code, $5.code) ;
        } else {
            sprintf(temp, "(return-from %s %s)", nombre_funcion, $2.code) ;
        }
    } else {
        sprintf(temp, "%s", $2.code) ;
    }
    }
    { $$code = gen_code(temp) ; }
    { $$code = gen_code("") ; }
;

abre_rama: /* lambda */ {nivel_rama++ ;}

;

bucle_while: WHILE '(' expresion ')' '{' abre_rama cuerpo '}'
    { nivel_rama-- ;
      sprintf (temp, "(loop while %s do\n%s)", $3.code, $7.code) ;
      $$code = gen_code(temp) ; }
;

```

```

bucle_for:   FOR '(' inicializ ';' expresion ';' oper_for ')' '{' abre_rama cuerpo '}'
              { nivel_rama--;
                sprintf(temp, "%s\n(loop while %s do\n%s\n%s)", $3.code, $5.code, $11.code, $7.code) ;
                $$code = gen_code(temp) ; }

;

inicializ: IDENTIF '=' expresion
              { if (en_funcion && es_local($1.code))
                sprintf(temp, "(setf %s %s)", nombre_local($1.code), $3.code) ;
                else
                  sprintf(temp, "(setf %s %s)", $1.code, $3.code) ;
                $$code = gen_code(temp) ; }

| INTEGER IDENTIF '=' expresion
              { insertar_local($2.code) ;
                sprintf(temp, "(setq %s %s)", nombre_local($2.code), $4.code) ;
                $$code = gen_code(temp) ;}

;

oper_for:   INC '(' IDENTIF ')'
              { if (en_funcion && es_local($3.code)) {
                sprintf(temp, "(setf %s (+ %s 1))", nombre_local($3.code), nombre_local($3.code)) ;
                } else {
                  sprintf(temp, "(setf %s (+ %s 1))", $3.code, $3.code) ;
                }
                $$code = gen_code(temp) ; }

| DEC '(' IDENTIF ')'
              { if (en_funcion && es_local($3.code)) {
                sprintf(temp, "(setf %s (- %s 1))", nombre_local($3.code), nombre_local($3.code)) ;
                } else {
                  sprintf(temp, "(setf %s (- %s 1))", $3.code, $3.code) ;
                }
                $$code = gen_code(temp) ; }

;

control_if:  IF '(' expresion ')' '{' abre_rama cuerpo '}'
              { nivel_rama-- ;
                sprintf(temp, "(if %s\n%s)", $3.code, wrap_progn($7.code)) ;
                $$code = gen_code(temp) ; }

| IF '(' expresion ')' '{' abre_rama cuerpo '}' ELSE '{' abre_rama cuerpo '}'
              { nivel_rama -= 2 ;
                sprintf(temp, "(if %s\n%s\n%s)", $3.code, wrap_progn($7.code), wrap_progn($12.code)) ;
                $$code = gen_code(temp) ; }

;

control_switch: SWITCH '(' IDENTIF ')' '{' switch_cases '}'
              { sprintf(temp, "(case %s\n%s)", $3.code, $6.code) ;
                $$code = gen_code(temp) ; }

;

switch_cases: CASE NUMBER ':' abre_rama cuerpo BREAK ';' switch_cases
              { nivel_rama-- ;
                sprintf(temp, "(%d\n%s)\n%s", $2.value, $5.code, $8.code) ;
                $$code = gen_code(temp) ; }

| DEFAULT ':' abre_rama cuerpo BREAK ';'
              { nivel_rama-- ;
                sprintf(temp, "(otherwise\n%s)", $4.code) ;
                $$code = gen_code(temp) ; }

|
              { $$code = gen_code("") ; }

;

sentencia:  INTEGER IDENTIF '=' expresion
              { if (en_funcion) {
                insertar_local($2.code) ;
                sprintf (temp, "(setq %s %s)", nombre_local($2.code), $4.code) ;
                } else {
                  sprintf (temp, "(setq %s %s)", $2.code, $4.code) ;
                }
                $$code = gen_code (temp) ; }

| PRINTF '(' STRING ',' elemento mult_elementos ')'
              { if (strlen($6.code) > 0) {
                sprintf (temp, "(princ %s)\n%s", $5.code, $6.code) ;
                } else {

```

```

        sprintf (temp, "(princ %s)", $5.code) ;
    }
    $$code = gen_code (temp) ; }
| INTEGER IDENTIF { if (en_funcion) {
    insertar_local($2.code);
    sprintf (temp, "(setq %s 0)", nombre_local($2.code)) ;
} else {
    sprintf (temp, "(setq %s 0)", $2.code) ;
}
    $$code = gen_code (temp) ; }
| INTEGER IDENTIF mult_assign { if (en_funcion) {
    insertar_local($2.code);
    sprintf (temp, "(setq %s 0) %s", nombre_local($2.code), $3.code) ;
} else {
    sprintf (temp, "(setq %s 0) %s", $2.code, $3.code) ;
}
    $$code = gen_code (temp) ; }
| INTEGER IDENTIF '=' expresion mult_assign { if (en_funcion) {
    insertar_local($2.code);
    sprintf (temp, "(setq %s %s) %s", nombre_local($2.code), $4.code, $5.code) ;
} else {
    sprintf (temp, "(setq %s %s) %s", $2.code, $4.code, $5.code) ;
}
    $$code = gen_code (temp) ; }
| IDENTIF '=' expresion { if (en_funcion && es_local($1.code)) {
    sprintf (temp, "(setf %s %s)", nombre_local($1.code), $3.code) ;
} else {
    sprintf (temp, "(setf %s %s)", $1.code, $3.code) ;
}
    $$code = gen_code(temp) ; }
| PUTS '(' STRING ')' { sprintf(temp, "(print \"%s\")", $3.code) ;
    $$code = gen_code (temp) ; }
| IDENTIF '(' lista_params ')' { sprintf(temp, "(%s %s)", $1.code, $3.code) ;
    $$code = gen_code(temp) ; }
| INTEGER IDENTIF '[' NUMBER ']' { if (en_funcion) {
    insertar_local($2.code) ;
    sprintf(temp, "(setq %s (make-array %d))", nombre_local($2.code), $4.value) ;
} else {
    sprintf(temp, "(setq %s (make-array %d))", $2.code, $4.value) ;
}
    $$code = gen_code(temp) ; }
| IDENTIF '[' expresion ']' '=' expresion { if (en_funcion && es_local($1.code)) {
    sprintf(temp, "(setf (aref %s %s) %s)", nombre_local($1.code), $3.code, $6.code) ;
} else {
    sprintf(temp, "(setf (aref %s %s) %s)", $1.code, $3.code, $6.code) ;
}
    $$code = gen_code(temp) ; }

;

lista_params: expresion r_lista_params { sprintf(temp, "%s %s", $1.code, $2.code) ;
    $$code = gen_code(temp) ; }
| { $$code = gen_code("") ; }
;

r_lista_params: ',' expresion r_lista_params { sprintf(temp, "%s %s", $2.code, $3.code) ;
    $$code = gen_code(temp) ; }
| { $$code = gen_code("") ; }
;

elemento: expresion { $$code = $1.code ; }
| STRING { sprintf(temp, "\"%s\"", $1.code) ;
    $$code = gen_code(temp) ; }
;

mult_elementos: ',' elemento mult_elementos { if (strlen($3.code) > 0) {

```

```

        sprintf(temp, "(princ %s)\n%s", $2.code, $3.code) ;
    } else {
        sprintf(temp, "(princ %s)", $2.code) ;
    }
    $$code = gen_code(temp) ; }
{ $$code = gen_code("") ; }

|
;

mult_asign:  ',' IDENTIF '=' expression mult_asign { if (en_funcion) {
    insertar_local($2.code) ;
    sprintf (temp, "(setq %s %s) %s", nombre_local($2.code), $4.code, $5.code) ;
} else {
    sprintf (temp, "(setq %s %s) %s", $2.code, $4.code, $5.code) ;
}
    $$code = gen_code (temp) ; }
| ',' IDENTIF mult_asign { if (en_funcion) {
    insertar_local($2.code) ;
    sprintf (temp, "(setq %s 0) %s", nombre_local($2.code), $3.code) ;
} else {
    sprintf (temp, "(setq %s 0) %s", $2.code, $3.code) ;
}
    $$code = gen_code (temp) ; }
| ',' IDENTIF '=' expression { if (en_funcion) {
    insertar_local($2.code) ;
    sprintf (temp, "(setq %s %s)", nombre_local($2.code), $4.code) ;
} else {
    sprintf (temp, "(setq %s %s)", $2.code, $4.code) ;
}
    $$code = gen_code (temp) ; }
| ',' IDENTIF { if (en_funcion) {
    insertar_local($2.code) ;
    sprintf (temp, "(setq %s 0)", nombre_local($2.code)) ;
} else {
    sprintf (temp, "(setq %s 0)", $2.code) ;
}
    $$code = gen_code (temp) ; }
;

expression:  termino { $$ = $1 ; }
| expression '+' expression { sprintf (temp, "(+ %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression '-' expression { sprintf (temp, "(- %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression '*' expression { sprintf (temp, "(* %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression '/' expression { sprintf (temp, "(/ %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression AND expression { sprintf (temp, "(and %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression OR expression { sprintf (temp, "(or %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression NEQ expression { sprintf (temp, "(/= %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression EQ expression { sprintf (temp, "(= %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression '<' expression { sprintf (temp, "(< %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression LTEQ expression { sprintf (temp, "(<= %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression '>' expression { sprintf (temp, "(> %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression GTEQ expression { sprintf (temp, "(>= %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }
| expression '%' expression { sprintf (temp, "(mod %s %s)", $1.code, $3.code) ;
    $$code = gen_code (temp) ; }

```

```

;

termino:    operando                { $$ = $1 ; }
          | '+' operando %prec UNARY_SIGN { $$ = $2 ; }
          | '-' operando %prec UNARY_SIGN { sprintf (temp, "(- %s)", $2.code) ;
                                           $$code = gen_code (temp) ; }
          | '!' operando %prec '!'      { sprintf (temp, "(not %s)", $2.code) ;
                                           $$code = gen_code (temp) ; }
;

operando:   IDENTIF                { if (en_funcion && es_local($1.code))
                                   sprintf(temp, "%s", nombre_local($1.code)) ;
                                   else
                                   sprintf(temp, "%s", $1.code) ;
                                   $$code = gen_code(temp) ; }
          | NUMBER                { sprintf (temp, "%d", $1.value) ;
                                   $$code = gen_code (temp) ; }
          | '(' expresion ')'      { $$ = $2 ; }
          | IDENTIF '(' lista_params ')'
                                   { sprintf(temp, "(%s %s)", $1.code, $3.code) ;
                                   $$code = gen_code(temp) ; }
          | IDENTIF '[' expresion ']' { if (en_funcion && es_local($1.code)) {
                                   sprintf(temp, "(aref %s %s)", nombre_local($1.code), $3.code) ;
                                   } else {
                                   sprintf(temp, "(aref %s %s)", $1.code, $3.code) ;
                                   }
                                   $$code = gen_code(temp) ; }
;

```

```

%%                                // SECCION 4    Codigo en C

```

```

int n_line = 1 ;

```

```

void insertar_local(char *nombre) {
    vars_locales[n_vars_locales++] = gen_code(nombre);
}

```

```

int es_local(char *nombre) {
    int i;
    for (i = 0; i < n_vars_locales; i++) {
        if (strcmp(vars_locales[i], nombre) == 0) return 1;
    }
    return 0;
}

```

```

void limpiar_locales() {
    n_vars_locales = 0;
}

```

```

char *nombre_local(char *var) {
    static char buf[512];
    sprintf(buf, "%s_%s", nombre_funcion, var);
    return buf;
}

```

```

char *wrap_progn(char *codigo) {
    static char buf[4096];
    if (strchr(codigo, '\n') != NULL) {
        sprintf(buf, "(progn\n%s)", codigo);
    } else {
        sprintf(buf, "%s", codigo);
    }
    return gen_code(buf);
}

```

```

int yyerror (char* mensaje) {
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
    printf ( "\n" ) ; // bye
}

char *int_to_string (int n) {
    char ltemp [2048] ;

    sprintf (ltemp, "%d", n) ;

    return gen_code (ltemp) ;
}

char *char_to_string (char c) {
    char ltemp [2048] ;

    sprintf (ltemp, "%c", c) ;

    return gen_code (ltemp) ;
}

char *my_malloc (int nbytes) { // reserva n bytes de memoria dinamica
    char *p ;
    static long int nb = 0 ;      // sirven para contabilizar la memoria
    static int nv = 0 ;          // solicitada en total

    p = malloc (nbytes) ;
    if (p == NULL) {
        fprintf (stderr, "No queda memoria para %d bytes mas\n", nbytes) ;
        fprintf (stderr, "Reservados %ld bytes en %d llamadas\n", nb, nv) ;
        exit (0) ;
    }
    nb += (long) nbytes ;
    nv++ ;

    return p ;
}

/*****
/***** Seccion de Palabras Reservadas *****/
/*****/

typedef struct s_keyword { // para las palabras reservadas de C
    char *name ;
    int token ;
} t_keyword ;

t_keyword keywords [] = {          // define las palabras reservadas y los
    "main",      MAIN,              // y los token asociados
    "int",        INTEGER,
    "while",      WHILE,
    "switch",     SWITCH,
    "case",       CASE,
    "break",      BREAK,
    "default",    DEFAULT,
    "puts",       PUTS,
    "printf",     PRINTF,
    "||",         OR,
    "&&",         AND,
    "==",         EQ,
    "!=",         NEQ,
    ">=",         GTEQ,
    "<=",         LTEQ,

```



```

        "if",      IF,
        "else",    ELSE,
        "for",     FOR,
        "inc",     INC,
        "dec",     DEC,
        "return",  RETURN,
        NULL,      0          // para marcar el fin de la tabla
    } ;

t_keyword *search_keyword (char *symbol_name)
{
    // Busca n_s en la tabla de pal. res.
    // y devuelve puntero a registro (simbolo)

    int i ;
    t_keyword *sim ;

    i = 0 ;
    sim = keywords ;
    while (sim [i].name != NULL) {
        if (strcmp (sim [i].name, symbol_name) == 0) {
            // strcmp(a, b) devuelve == 0 si a==b
            return &(sim [i]) ;
        }
        i++ ;
    }

    return NULL ;
}

/*****
***** Seccion del Analizador Lexicografico *****/
*****/

char *gen_code (char *name)    // copia el argumento a un
{                               // string en memoria dinamica
    char *p ;
    int l ;

    l = strlen (name)+1 ;
    p = (char *) my_malloc (l) ;
    strcpy (p, name) ;

    return p ;
}

int yylex ()
{
    // NO MODIFICAR ESTA FUNCION SIN PERMISO
    int i ;
    unsigned char c ;
    unsigned char cc ;
    char ops_expandibles [] = "!<=>%&/+.*" ;
    char temp_str [256] ;
    t_keyword *symbol ;

    do {
        c = getchar () ;

        if (c == '#') { // Ignora las lineas que empiezan por # (#define, #include)
            do { // OJO que puede funcionar mal si una linea contiene #
                c = getchar () ;
            } while (c != '\n') ;
        }
    }
}

```

```

if (c == '/') { // Si la linea contiene un / puede ser inicio de comentario
    cc = getchar () ;
    if (cc != '/') { // Si el siguiente char es / es un comentario, pero...
        ungetc (cc, stdin) ;
    } else {
        c = getchar () ; // ...
        if (c == '@') { // Si es la secuencia //@ ==> transcribimos la linea
            do { // Se trata de codigo inline (Codigo embebido en C)
                c = getchar () ;
                putchar (c) ;
            } while (c != '\n') ;
        } else { // ==> comentario, ignorar la linea
            while (c != '\n') {
                c = getchar () ;
            }
        }
    }
} else if (c == '\\') c = getchar () ;

if (c == '\n')
    n_line++ ;

} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;

if (c == '\"') {
    i = 0 ;
    do {
        c = getchar () ;
        temp_str [i++] = c ;
    } while (c != '\"' && i < 255) ;
    if (i == 256) {
        printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line) ;
    } // habria que leer hasta el siguiente " , pero, y si falta?
    temp_str [--i] = '\0' ;
    yylval.code = gen_code (temp_str) ;
    return (STRING) ;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc (c, stdin) ;
    scanf ("%d", &yylval.value) ;
    // printf ("\nDEV: NUMBER %d\n", yylval.value) ; // PARA DEPURAR
    return NUMBER ;
}

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    i = 0 ;
    while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9') || c == '_') && i < 255) {
        temp_str [i++] = tolower (c) ;
        c = getchar () ;
    }
    temp_str [i] = '\0' ;
    ungetc (c, stdin) ;

    yylval.code = gen_code (temp_str) ;
    symbol = search_keyword (yylval.code) ;
    if (symbol == NULL) { // no es palabra reservada -> identificador antes vvariable
        // printf ("\nDEV: IDENTIF %s\n", yylval.code) ; // PARA DEPURAR
        return (IDENTIF) ;
    } else {
        // printf ("\nDEV: OTRO %s\n", yylval.code) ; // PARA DEPURAR
        return (symbol->token) ;
    }
}
}

```

```

    if (strchr (ops_expandibles, c) != NULL) { // busca c en ops_expandibles
        cc = getchar () ;
        sprintf (temp_str, "%c%c", (char) c, (char) cc) ;
        symbol = search_keyword (temp_str) ;
        if (symbol == NULL) {
            ungetc (cc, stdin) ;
            yylval.code = NULL ;
            return (c) ;
        } else {
            yylval.code = gen_code (temp_str) ; // aunque no se use
            return (symbol->token) ;
        }
    }

    // printf ("\nDEV: LITERAL %d #c#\n", (int) c, c) ; // PARA DEPURAR
    if (c == EOF || c == 255 || c == 26) {
        // printf ("tEOF ") ; // PARA DEPURAR
        return (0) ;
    }

    return c ;
}

int main ()
{
    yyparse () ;
}

```